

Saturation for a general class of models

Andrew S. Miner

Department of Computer Science
Iowa State University, Ames, Iowa, 50011
E-mail: asminer@iastate.edu

Abstract

Implicit techniques for construction and representation of the reachability set of a high-level model have become quite efficient for certain types of models. In particular, previous work developed a “saturation” algorithm that exploits asynchronous behavior to efficiently construct the reachability set using multiway decision diagrams, but requires each model event to be expressible as a Kronecker product. In this paper, we develop a new version of the saturation algorithm that works for a general class of models: models whose events are not necessarily expressible as Kronecker products, models containing events with complex priority structures, and models whose state variables have unknown bounds. We apply our algorithm to several examples and give detailed experimental results.

1. Introduction

The generation of the reachability set for a discrete-state model is a necessary step and a significant challenge in many types of studies. Formal verification techniques, including model checking [10], may require reachability set construction to verify some desirable property about a system, such as absence of deadlock. In performance evaluation studies, the reachability set is often required to generate a Markov reward model, which in turn is analyzed to produce performance measures of interest. In either case, reachability set generation can be difficult in practice due to the state-space explosion problem: the reachability set can be *extremely* large, even for a “small” model. Explicit methods are impractical for such complex systems.

Approaches based on implicit methods (also known as *symbolic* methods), which often have space and time complexity much less than the size of the reachability set, have been embraced by many researchers. Decision diagrams, in particular binary decision diagrams (BDDs) [1] and multiway decision diagrams (MDDs) [12], are widely accepted as efficient and compact data structures for the representa-

tion of extremely large sets. While decision diagrams degrade to an explicit approach in the worst case, they have been used successfully in practice [4, 5, 7, 16, 17].

To use decision diagrams to represent sets of states, each state is decomposed into a collection of K substates, corresponding to levels of nodes in the decision diagram. In an asynchronous system, a given state-to-state transition usually does not affect all K substates; some substates do not affect and are not changed by the transition. The *saturation* algorithm exploits this property to generate reachability sets extremely efficiently for asynchronous systems [7], but originally imposed three constraints: first, that the possible substates for component k (or a reasonable superset) are known *a priori*; second, that actions have equal priority; third, that the effect of an action on a component k depends only on substate k . The first and second restrictions are addressed (in isolation) in [8] and [15], respectively. The third restriction, known as the “Kronecker product” or “product-form” requirement, is the subject of this paper.

The remainder of the paper is organized as follows. Section 2 is a brief overview of background material. Section 3 describes how to eliminate the “product-form” requirement when the component substates and next-state function are already known, and Section 4 describes how to generate the reachability set, the next-state function, and the component substates simultaneously using saturation. Section 5 discusses implementation-related issues. Experimental results for several examples are given in Section 6. Finally, Section 7 concludes the work.

2. Background and definitions

Rather than use a specific high-level formalism, we specify a discrete-state model as a triple $(\hat{S}, \mathcal{N}, S^0)$, where

- $\hat{S}$ is the set of possible states of the model;
- $\mathcal{N} : \hat{S} \rightarrow 2^{\hat{S}}$ is the next-state function, where $\mathcal{N}(\mathbf{i})$ specifies the set of states that can be reached from state $\mathbf{i} \in \hat{S}$ in one step;
- $S^0 \subseteq \hat{S}$ is the non-empty set of initial states.

For purposes of this work, we assume that the set $\hat{\mathcal{S}}$ is finite. The reachability set $\mathcal{S}$ is the reflexive and transitive closure of $\mathcal{N}$, starting from the initial states:

$$\mathcal{S} = \mathcal{S}^0 \cup \mathcal{N}(\mathcal{S}^0) \cup \mathcal{N}^2(\mathcal{S}^0) \cup \dots = \mathcal{N}^*(\mathcal{S}^0),$$

where we extend the next-state function to apply also to sets of states, $\mathcal{N}(\mathcal{I}) = \bigcup_{\mathbf{i} \in \mathcal{I}} \mathcal{N}(\mathbf{i})$.

2.1. MDDs to represent subsets of $\hat{\mathcal{S}}$

We require that the model is composed of $K > 1$ components, with $\hat{\mathcal{S}} = \mathcal{S}_K \times \dots \times \mathcal{S}_1$ and $n_k = |\mathcal{S}_k|$. The case $K = 1$ is equivalent to explicit representation and generation, so we do not consider it further. The set $\mathcal{S}_k$ is usually referred to as the *local states* for component k . For notational convenience, we assume that $\mathcal{S}_k = \{0, \dots, n_k - 1\}$.

Any subset of $\hat{\mathcal{S}}$ can be represented using an MDD, defined as follows [12]. An MDD is a directed, acyclic graph consisting of m *terminal nodes*, which are labeled from integers $\{0, \dots, m - 1\}$, and *non-terminal nodes* p , which are labeled with a variable v_k , and contain a vector of n_k pointers to other nodes, denoted $p.\text{down}$. We use the shorthand $p[i]$ to denote element i of $p.\text{down}$. In this work, we assume all MDDs are *ordered*: all nodes with variable v_k are said to belong to *level* k , terminal nodes are said to belong to level 0, and all pointers from a level- k node are to nodes at a level less than k . The level of a node p is denoted $\text{Top}(p)$. A given state $\mathbf{i} = (i_K, \dots, i_1)$ is contained in a subset represented by a K -level MDD iff the path through the MDD, starting at the level- K node, following downward pointer i_k at level k , reaches terminal node 1.

MDDs remain compact through the use of reductions. Two level- k nodes p and p' are said to be *duplicates* if $p.\text{down} = p'.\text{down}$. A level- k node p is said to be *redundant* if $p[i_k] = p[0]$ for all $0 \leq i_k < n_k$. An MDD is *reduced* if it contains no duplicates or redundant nodes. An MDD is *quasi-reduced* if it contains no duplicates, and every path from the level- K node visits all other levels. It can be shown that both reduced and quasi-reduced (ordered) MDDs are canonical representations. Set operations such as union, difference, and intersection can be performed directly on their MDD representations [16]; the complexity of these operations is based on the number of nodes in the MDDs, rather than the number of elements in the sets encoded by the MDDs. For the remainder of the paper, we assume all MDDs are quasi-reduced and ordered.

As a simple example, a small Petri net model is shown in Figure 1(a). The model has $K = 4$ components, where the local state of component k corresponds to the number of tokens in place p_k . The reachability set, encoded as an MDD, is shown in Figure 1(b) for the case $N = 2$, which has initial state $(0, 0, 0, 2)$. Paths that lead only to terminal node 0 are omitted for clarity.

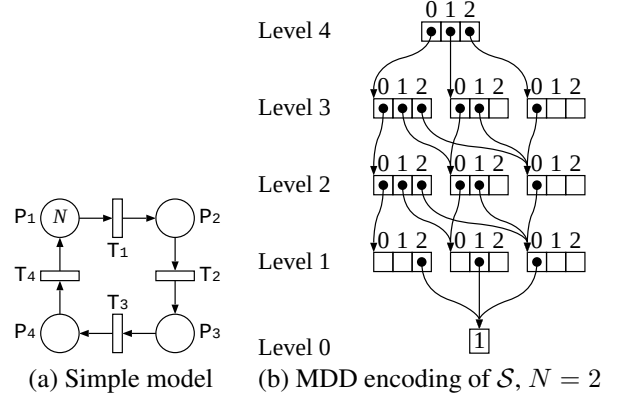


Figure 1. A model and its reachability set

2.2. Matrix diagrams for next-state functions

Like MDDs, a matrix diagram (MxD) is a directed, acyclic graph with terminal and non-terminal nodes; however, for each non-terminal node p with variable v_k , $p.\text{down}$ is a matrix of $n_k \times n_k$ pointers to other nodes [15]. As with MDDs, we assume that all MxDs are ordered. Matrix diagrams can represent a next-state function $\mathcal{N}$ as follows. Given a pair of states $\mathbf{i} = (i_K, \dots, i_1)$ and $\mathbf{j} = (j_K, \dots, j_1)$, then $\mathbf{j} \in \mathcal{N}(\mathbf{i})$ iff the path through the MxD, starting at the level- K node, following downward pointer (i_k, j_k) at level k , reaches terminal node 1.

Matrix diagrams use a slightly different set of reduction rules than MDDs. A level- k node p is said to be an *identity node* if $p[i_k, i_k] = p[0, 0]$ for all $0 \leq i_k < n_k$, and $p[i_k, j_k] = 0$ for all $i_k \neq j_k$, while the definition of *duplicate nodes* is analogous to the MDD case. A matrix diagram is reduced if it contains no duplicates or identity nodes. Set union and set intersection of two next-state functions encoded as MxDs can be performed similarly to MDDs [15]. It is important to note that an identity node at level k corresponds to the case where component k does not affect and is unchanged in the next state reached along a particular path in the MxD. This type of behavior occurs frequently in asynchronous systems.

An alternative to the use of MxDs to represent a next-state function is to use a Kronecker expression [3, 13]. Informally, a Kronecker product is the special case of a MxD with at most one node per level, where all downward pointers from a non-terminal node are either to terminal node 0, or to one other node. A Kronecker expression can be used for a next-state function $\mathcal{N}$ iff it can be expressed as $\mathcal{N}(\mathbf{i}) = \bigcup_{e \in \mathcal{E}} \mathcal{N}^e(\mathbf{i})$ where $\mathcal{N}^e$ can be expressed as a Kronecker product for a particular *event* e of the model. That is, $\mathcal{N}^e(\mathbf{i}) = \mathcal{N}_K^e(i_K) \times \dots \times \mathcal{N}_1^e(i_1)$, where the cross products become Kronecker products if the set functions are instead represented as boolean matrices. In theory, it is always pos-

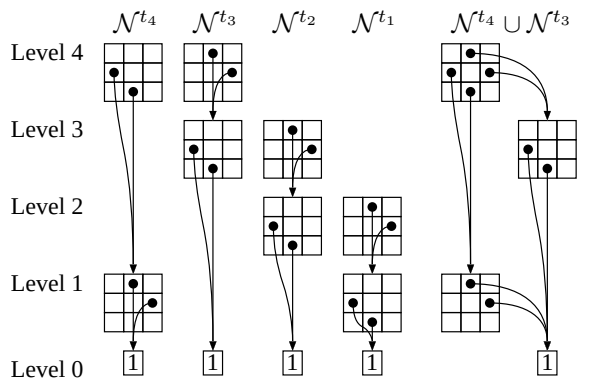


Figure 2. Next-state functions for events

sible to obtain a Kronecker expression: in the worst case, each function $\mathcal{N}^e$ will describe exactly one state-to-state transition: $\mathcal{N}^e(\mathbf{i}) = \{\mathbf{j}\}$, and $\mathcal{N}^e(\mathbf{i}') = \emptyset$ for all $\mathbf{i}' \neq \mathbf{i}$. For many types of formalisms, the set $\mathcal{E}$ corresponds naturally to formalism features; for instance, $\mathcal{E}$ might be the set of transitions of a Petri net.

Example MxDs corresponding to next-state functions for the simple model of Figure 1(a) are shown in Figure 2. Again, paths to terminal node 0 are omitted for clarity. Note that the next-state functions $\mathcal{N}^{t_4}$, $\mathcal{N}^{t_3}$, $\mathcal{N}^{t_2}$ and $\mathcal{N}^{t_1}$ are Kronecker products, while $\mathcal{N}^{t_4} \cup \mathcal{N}^{t_3}$ is not (it is the sum of Kronecker products).

2.3. Saturation

The saturation algorithm to construct the reachability set (as an MDD) [7] exploits the property of asynchronous systems that some state changes (i.e., events) do not affect particular components: if $\mathbf{j} \in \mathcal{N}(\mathbf{i})$, it is common that $i_k = j_k$ for many k . Thus, given a Kronecker expression for the next-state function, each event e may affect only a few components; any unaffected components will produce identity nodes in the MxD representation of $\mathcal{N}^e$. In particular, if an event affects component k , but does not affect components $(k+1)$ through K , then the topmost MxD node for $\mathcal{N}^e$ will be a level- k node. The saturation algorithm [7] partitions the next-state function $\mathcal{N}$ into $\mathcal{N}_K, \dots, \mathcal{N}_1$, where $\mathcal{N}_k$ is the union of all $\mathcal{N}^e$ with $\text{Top}(\mathcal{N}^e) = k$. During reachability set generation, any newly-created level- k MDD node is “saturated”, by repeatedly considering all transitions in $\mathcal{N}_k$ until no new states are reached.

As originally presented in [7], the saturation algorithm requires a Kronecker expression for each $\mathcal{N}_k$ as input. This implies the following:

1. Sets $\mathcal{S}_k$ and next-state functions $\mathcal{N}^e$ are known.
2. All events have equal priority.
3. For all $e \in \mathcal{E}$, $\mathcal{N}^e$ is a Kronecker product.

The saturation algorithm was modified in [8] to address requirement (1) by updating sets $\mathcal{S}_k$ and next-state functions $\mathcal{N}^e$ with new local states *during* generation. Requirement (2) was addressed in [15], for cases where the functions $\mathcal{N}^e$ would be Kronecker products if priorities are ignored, by first constructing MxDs for each $\mathcal{N}^e$ from the Kronecker product and then correcting the MxDs to take priorities into account. While [7] uses a next-state function for each event, [15] instead constructs a single next-state function $\mathcal{N}$ (by taking the union of each $\mathcal{N}^e$), splits $\mathcal{N}$ into $\mathcal{N}_K, \dots, \mathcal{N}_1$ automatically (based on matrix diagram structure), and then uses a modified saturation algorithm that works with MxDs.

Requirement (3) was addressed recently [11] via the use of an iterative algorithm (not saturation) that performs two steps per iteration, a generalization of the approach in [16]: events that affect a single component are handled efficiently in a special way, then all other events are handled. The differences between these approaches and saturation [7] can be seen when all events affect more than one component: if each event affects only a few components, the saturation algorithm is still able to exploit asynchronous behavior, as long as some events have $\text{Top}(\mathcal{N}^e) < K$. To the best of the author’s knowledge, the saturation algorithm has not yet been applied successfully to models where requirement (3) does not hold, except by splitting events or merging components until the requirement holds.

Note that the simple model of Figure 1(a) satisfies the Kronecker-product requirement using $K = 4$ components (one for each place) and events $\mathcal{E} = \{t_4, t_3, t_2, t_1\}$, since the next-state functions $\mathcal{N}^{t_4}, \mathcal{N}^{t_3}, \mathcal{N}^{t_2}, \mathcal{N}^{t_1}$ are Kronecker products. However, if all transitions are made “flushing” (e.g., the arcs from p_1 to t_1 and t_1 to p_2 now have cardinality $\#(p_1)$ instead of cardinality one, etc.), requirement (3) no longer holds: the new number of tokens in p_2 after t_1 fires depends on the number of tokens in p_1 . For this example, to satisfy requirement (3) by merging components, all components will need to be merged into one large component (with $K = 1$). The alternative is to split each transition into N events, one event for each possible (nonzero) number of tokens in the input place of the transition. As we will show in Section 6, the latter approach can significantly increase computation time.

3. Lifting the Kronecker product requirement

3.1. Constructing the next-state function

For simplicity, we assume (for now) that the other two requirements hold (e.g., local state spaces $\mathcal{S}_k$ are known, and all events have equal priority). For each event e , we express the next-state function $\mathcal{N}^e$ as

$$\mathcal{N}^e = \mathcal{N}_{\mathcal{G}_n}^e \times \dots \times \mathcal{N}_{\mathcal{G}_1}^e \quad (1)$$

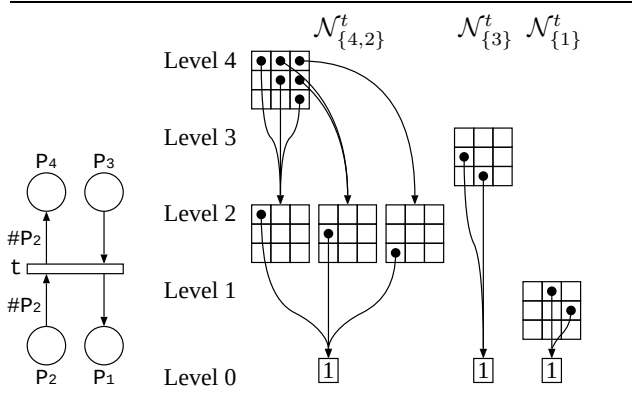


Figure 3. Next-state functions by group

where $\mathcal{G}_g^e$ is a group of components, $\mathcal{G}_n^e, \dots, \mathcal{G}_1^e$ is a partition of the set of components, and function $\mathcal{N}_{\mathcal{G}_g^e}^e$ cannot be broken into a cross product of functions with smaller groups. A component not affected by event e will be in its own group $\mathcal{G}$, and $\mathcal{N}_{\mathcal{G}}^e$ will be the identity function. Note that if $\mathcal{N}^e$ satisfies the Kronecker-product requirement, then each group will consist of a single component. Otherwise, components that violate the requirement are grouped together (as suggested in [7, 16]); however, components are grouped only for the necessary events. That is, the partition $\mathcal{H}^e = \{\mathcal{G}_n^e, \dots, \mathcal{G}_1^e\}$ for a particular event e may differ from the partition for a different event. For an event e , its partition $\mathcal{H}^e$ can be determined automatically by examining the high-level model: initially, there are K groups, with $\mathcal{G}_k = \{k\}$; for each complex (not product-form) dependency between components k_1 and k_2 , the groups containing k_1 and k_2 are merged.

As an example, consider Petri net transition t shown in Figure 3. Since the arc from t to p_4 depends on the state of p_2 (a dependency that violates product form), components 4 and 2 must be grouped together. Thus, the partition for event t is $\mathcal{H}^t = \{\{4, 2\}, \{3\}, \{1\}\}$.

Once the partition $\mathcal{H}^e$ for a given event e has been determined, we construct the MxD for functions $\mathcal{N}_{\mathcal{G}}^e$ for each $\mathcal{G} \in \mathcal{H}^e$. This is done by enumerating (explicitly) the partial states in the set $\mathcal{S}_{\mathcal{G}} = \mathcal{S}_{k_g} \times \dots \times \mathcal{S}_{k_1}$, where $\mathcal{G} = \{k_1, \dots, k_g\}$. Clearly, the cost of enumerating $\mathcal{S}_{\mathcal{G}}$ grows dramatically with the number of components in the group $\mathcal{G}$; in the worst case, $\mathcal{G} = \{1, \dots, K\}$ and $\mathcal{S}_{\mathcal{G}} = \hat{\mathcal{S}}$, and the approach becomes explicit. The enumeration can often be accelerated by ignoring partial states in which event e definitely cannot occur: if event e cannot occur when component k_g is in local state s_{k_g} , then partial states $\{s_{k_g}\} \times \mathcal{S}_{k_{g-1}} \times \dots \times \mathcal{S}_{k_1}$ do not need to be considered. For each partial state $\mathbf{i} \in \mathcal{S}_{\mathcal{G}}$ that does not disable event e , we must determine the partial states $\mathbf{j} \in \mathcal{S}_{\mathcal{G}}$ that can be reached from $\mathbf{i}$ when event e occurs, considering only the

components in $\mathcal{G}$. Each such pair $(\mathbf{i}, \mathbf{j})$ is a “partial” transition which must be represented by the function $\mathcal{N}_{\mathcal{G}}^e$. These pairs are inserted into the MxD for $\mathcal{N}_{\mathcal{G}}^e$, one at a time, using algorithm Insert shown in Figure 4, which is a simplified version of the AddEntry algorithm presented in [14]. The purpose of line 8 will be explained in Section 3.2.

Further considering event t and group $\mathcal{G} = \{4, 2\}$ as shown in Figure 3, if we assume that the local state spaces are $\mathcal{S}_4 = \mathcal{S}_2 = \{0, 1, 2\}$, then transition t might occur in all partial states $\mathbf{i} \in \mathcal{S}_4 \times \mathcal{S}_2$. For a particular partial state such as $\mathbf{i} = (1, 1)$, it can be determined that the occurrence of t might lead to partial state $\mathbf{j} = (2, 0)$. Note that the pair $(1, 1), (2, 0)$ is represented by the MxD $\mathcal{N}_{\{4, 2\}}^t$ shown in Figure 3, which can be seen by following pointer $(1, 2)$ from the level-4 matrix and pointer $(1, 0)$ from the level-2 matrix to reach terminal node 1.

A cross product operator is necessary to construct the overall next-state function for a given event from its per-group next-state functions. The cross product of two functions A and B represented as MxDs is simple to construct if the top-most level of B is below the bottom-most level of A : all pointers to terminal node 1 in A are directed to the root node of B . If the levels of A and B are “interleaved”, then their cross product can be constructed recursively, using algorithm CrossProd shown in Figure 4, which works as follows. Assuming $Top(A) > Top(B)$ and $Top(A) = k$, then $C = A \times B$ is obtained by constructing a new level- k node whose downward pointers are to the cross product of B and the corresponding downward pointers in the level- k node in A (line 8). The recursion terminates when either A or B is a terminal node (lines 2–3).

As is common with binary operations on decision diagrams [2], algorithm CrossProd generates a reduced graph and utilizes a “cache”. Reduction is performed in line 10 (procedure Reduce) by checking newly-created node C for duplicates in a uniqueness table: if a duplicate is found, it is returned and node C is deleted; otherwise, C is added to the table and returned. The cache (implemented as a hash table) prevents duplicate computations: after computing $C = A \times B$, this information is added to the operation cache *opCache* (line 11); future requests to compute $A \times B$ will use the cached result (line 4). Using a cache in this way reduces computational complexity from exponential in the number of levels (in the worst case) to the product of the sizes (i.e., number of nodes) of the operands [1].

3.2. Constructing local state spaces

The above assumption that the local state spaces $\mathcal{S}_k$ can be constructed in isolation, before constructing the next-state function or the reachability set, is not always realistic, especially considering models that violate the Kronecker product requirement. Local state spaces can be constructed

<pre> PreGenerate(out : set $\mathcal{S}_K, \dots, \mathcal{S}_1$, MxD $\mathcal{N}_K, \dots, \mathcal{N}_1$) 1: for each $k \in \{1, \dots, K\}$ do 2: $\hat{\mathcal{S}}_k \leftarrow \mathcal{S}_k^0$; $\mathcal{U}_k \leftarrow \mathcal{S}_k^0$; $\mathcal{S}_k \leftarrow \emptyset$; 3: end for 4: while $\exists k' \in \{1, \dots, K\} : \mathcal{U}_{k'} \neq \emptyset$ do 5: UpdateLevel(k'); 6: for each $k \in \{1, \dots, K\}$ do 7: $\mathcal{U}_k \leftarrow \hat{\mathcal{S}}_k \setminus \mathcal{S}_k$; 8: end for 9: end while 10: BuildAll($\mathcal{N}_K, \dots, \mathcal{N}_1$); </pre>	<pre> Insert(in : MxD $\mathcal{N}$, group $\mathcal{G}$, state i, j) 1: while $\mathcal{G} \neq \emptyset$ do 2: $k \leftarrow \max(\mathcal{G})$; $\mathcal{G} \leftarrow \mathcal{G} \setminus \{k\}$; 3: if $\mathcal{G} = \emptyset$ then $\mathcal{N}[i_k, j_k] \leftarrow 1$; 4: if $\mathcal{N}[i_k, j_k] = 0$ then 5: $\mathcal{N}[i_k, j_k] \leftarrow$ new level-max($\mathcal{G}$) node; 6: end if 7: $\mathcal{N} \leftarrow \mathcal{N}[i_k, j_k]$; 8: $\hat{\mathcal{S}}_k \leftarrow \hat{\mathcal{S}}_k \cup \{j_k\}$; 9: end while </pre>
<pre> UpdateLevel(in : level k) • Explore $\mathcal{U}_k$ and update next-state functions. 1: for each event $e \in \mathcal{E}$ that affects component k do 2: Find g such that $k \in \mathcal{G}_g^e$; 3: UpdateNext($e, \mathcal{G}_g^e, k$); 4: $\mathcal{N}_{\mathcal{G}_g^e}^e \leftarrow \text{Reduce}(\mathcal{N}_{\mathcal{G}_g^e}^e)$; 5: end for 6: $\mathcal{S}_k \leftarrow \mathcal{S}_k \cup \mathcal{U}_k$; </pre>	<pre> UpdateNext(in : event e, group $\mathcal{G}$, level k) • Update $\mathcal{N}_{\mathcal{G}}^e$ for new local states in component k. 1: $\mathcal{G}' \leftarrow \mathcal{G} \setminus \{k\}$; 2: for each $i \in \mathcal{S}_{\mathcal{G}'}$ $\times \mathcal{U}_k$ do 3: for each j : MightReach(e, i, j) do 4: Insert($\mathcal{N}_{\mathcal{G}}^e, \mathcal{G}, i, j$); 5: end for 6: end for </pre>
<pre> BuildAll(out : MxD $\mathcal{N}_K, \dots, \mathcal{N}_1$) 1: for each $e \in \mathcal{E}$ do 2: $\mathcal{N}^e \leftarrow 1$; 3: for each $\mathcal{G} \in \mathcal{H}^e$ do 4: $\mathcal{N}^e \leftarrow \text{CrossProd}(\mathcal{N}^e, \mathcal{N}_{\mathcal{G}}^e)$; 5: end for 6: end for 7: Adjust each $\mathcal{N}^e$ for priorities; 8: $\mathcal{N} = \bigcup_{e \in \mathcal{E}} \mathcal{N}^e$; 9: Split $\mathcal{N}$ into $\mathcal{N}_K, \dots, \mathcal{N}_1$ </pre>	<pre> CrossProd(in : MxD A, MxD B) 1: if Top(A) < Top(B) then Swap(A, B); 2: if $B = 0$ then return 0; 3: if $B = 1$ then return A; 4: if $\exists C : (A, \times, B, C) \in \text{opCache}$ then return C; 5: $k \leftarrow \text{Top}(A)$; 6: $C \leftarrow$ new level-k MxD node; 7: for each $i_k, j_k \in \mathcal{S}_k$ do 8: $C[i_k, j_k] \leftarrow \text{CrossProd}(A[i_k, j_k], B)$; 9: end for 10: $C \leftarrow \text{Reduce}(C)$; 11: $\text{opCache} \leftarrow \text{opCache} \cup (A, \times, B, C)$; 12: return C; </pre>

Figure 4. Generating next-state functions and local state spaces simultaneously

during construction of the next-state functions by allowing the addition of any newly-discovered local states. The initial local states $\mathcal{S}_k^0$ can be determined as the subset of local states $\mathcal{S}_k$ appearing in $\mathcal{S}^0$, so that $\mathcal{S}^0 \subseteq \mathcal{S}_K^0 \times \dots \times \mathcal{S}_1^0$. We then explicitly explore the groups for each event, adding local states as they are discovered. To this end, we maintain two additional sets of local states: $\hat{\mathcal{S}}_k$, which includes the initial states $\mathcal{S}_k^0$ and any local states that might be reached due to the occurrence of an event; and $\mathcal{U}_k$, which is the set of local states for component k that must still be explored. The set $\mathcal{S}_k$ refers to local states for component k that have been explored already: transitions out of those states are present in the next-state functions.

An algorithm to simultaneously construct local state spaces and next-state functions is shown in Figure 4, in procedure PreGenerate. Initially, the next-state functions are constructed using the sets $\mathcal{S}_k^0$ as the local states. Newly-discovered local states are added to sets $\hat{\mathcal{S}}_k$ in procedure Insert, via line 8. Whenever a component k has new local states, all events that depend on component k must update

their next-state functions. This is done by procedure UpdateLevel, which updates MxDs to include local states in $\mathcal{U}_k$ for every event e and group $\mathcal{G}$ such that $\mathcal{N}_{\mathcal{G}}^e$ depends on component k . Procedure UpdateNext performs this task for a particular $\mathcal{N}_{\mathcal{G}}^e$, by explicitly visiting the unexplored local states $\mathcal{U}_k$ along with the explored states for other components within group $\mathcal{G}$. The set of explored local states $\mathcal{S}_k$ is then updated. This process repeats until no new local states are discovered.

Note that this algorithm may generate a larger set $\mathcal{S}_k$ than can actually be reached, since the local states are generated considering only partial states within $\mathcal{S}_{\mathcal{G}}$ for some group $\mathcal{G}$ of components. That is, due to synchronizations between groups of components, some local states generated might never occur within $\mathcal{S}$. This can pose problems since the generated sets $\mathcal{S}_k$ can be significantly larger than the actual set of local states that can occur, and are not even guaranteed to be finite even for finite reachability sets $\mathcal{S}$. These problems can sometimes be addressed by modifying the components appropriately (e.g., by adding inhibitor arcs to a Petri net).

The alternative is to use a more dynamic method of constructing sets $\mathcal{S}_k$; this approach is considered in Section 4.

After the local state spaces $\mathcal{S}_k$ and the next-state functions for each group of each event have been constructed, procedure `BuildAll` is called, which first constructs the next-state function $\mathcal{N}^e$ for each event e by taking the cross product of the functions for each group of that event (lines 1–6). Once we have obtained MxDs encoding the next-state functions for each event, the techniques described in [15] are applied to construct the necessary MxDs for use with the saturation algorithm, as follows. First, the next-state functions for each event are corrected to include event priorities (line 7): if event e' has priority over e , then $\mathcal{N}^e$ must be modified so that event e cannot occur whenever event e' is enabled. Second, a single overall next-state function $\mathcal{N}$ is constructed as the union of each $\mathcal{N}^e$ (line 8). Finally, the next-state function $\mathcal{N}$ is split into $\mathcal{N}_K, \dots, \mathcal{N}_1$ (line 9). Note that each $\mathcal{N}_k$ is either zero or an MxD node with $\text{Top}(\mathcal{N}_k) = k$, and $\mathcal{N}_K \cup \dots \cup \mathcal{N}_1 = \mathcal{N}$. For more details, we refer the interested reader to [15].

3.3. Constructing the reachability set

An algorithm to perform saturation to construct the reachability set $\mathcal{S}$, given next-state functions $\mathcal{N}_K, \dots, \mathcal{N}_1$ encoded as matrix diagrams, is presented in [15]; this algorithm can be used without modification after procedure `BuildAll` is called. If the model contains immediate events, the set $\mathcal{S}$ will include both tangible and vanishing states. If the set of tangible reachable states $\mathcal{T}$ is desired, it can be determined as the subset of $\mathcal{S}$ in which no immediate events are enabled, using a simple query [15, 16].

4. On-the-fly construction

In the previous section, we assumed that the next-state function and local state spaces were constructed in a separate step before proceeding with the saturation algorithm. However, there may be cases where it is impossible to construct the local state spaces *a priori*, in particular if synchronizations with other components must be considered. We now describe the necessary modifications to the saturation algorithm so that the local state spaces, next-state function, and reachability set can be constructed simultaneously.

4.1. Updating local state spaces

Conceptually, our approach is similar to [8]; however, the removal of the product-form restriction leads to additional challenges. Consistent with [8], we use $\mathcal{S}_k$ to denote the set of *confirmed* local states for component k , which refers to local states that appear in at least one reachable (global)

state; and $\hat{\mathcal{S}}_k$ to denote the set of potentially-reachable local states for component k , which includes both confirmed states and *unconfirmed* states (those that do not yet appear in a reachable state). As in [8], our on-the-fly algorithm “confirms” a local state $s_k \in \hat{\mathcal{S}}_k$ only when a global state containing s_k is reached. Newly-confirmed local states s_k are added to $\mathcal{U}_k$, to be considered when the next-state functions are updated.

4.2. Updating next-state functions

During reachability set construction, the next-state functions describe possible transitions from states in $\mathcal{S}_K \times \dots \times \mathcal{S}_1$ to states in $\hat{\mathcal{S}}_K \times \dots \times \hat{\mathcal{S}}_1$. That is, every level- k MxD node considers transitions out of confirmed locals $\mathcal{S}_k$ only, but these transitions can lead to confirmed or unconfirmed locals $\hat{\mathcal{S}}_k$. These MxD nodes can be viewed either as *rectangular*, with dimension $|\mathcal{S}_k| \times |\hat{\mathcal{S}}_k|$, or equivalently as square with dimension $|\hat{\mathcal{S}}_k| \times |\hat{\mathcal{S}}_k|$ with rows $\hat{\mathcal{S}}_k \setminus \mathcal{S}_k$ containing pointers to terminal node zero. We use the latter conceptual view in the following discussion.

When a local state s_k is confirmed, the next-state functions must be updated. Specifically, row s_k of all level- k MxD nodes, which previously contained pointers to terminal node zero, must be updated to accurately capture transitions (if any) out of local state s_k . In [8], which uses a Kronecker product representation for the next-state function, this can be done by updating the appropriate level- k matrices, since the next-state function satisfies the product-form requirement. In our case, many levels of MxD nodes may need to be updated: if next-state function $\mathcal{N}_G^e$ depends on component k (i.e., $k \in \mathcal{G}$), then all levels in $\mathcal{G}$ must be checked in the MxD for $\mathcal{N}_G^e$; this can be done using procedure `UpdateLevel` shown in Figure 4.

After the individual next-state functions have been updated, functions $\mathcal{N}_K, \dots, \mathcal{N}_1$ must be reconstructed by calling procedure `BuildAll` shown in Figure 4. Unlike [8], which determines each $\mathcal{N}_k$ *a priori* based on the event structure in the high-level model, our approach determines each $\mathcal{N}_k$ dynamically based on the next-state function itself. As such, the addition of a confirmed local state s_k can change $\mathcal{N}_{k'}$ for any k' by both adding new transitions and removing transitions. This can occur (for instance) when s_k is the “first” local state to affect an event e : before confirming s_k , the next-state function $\mathcal{N}^e$ contains identity nodes at level k ; afterward, $\mathcal{N}^e$ contains at least one non-identity node at level k . In this case, the update can remove transitions from $\mathcal{N}_{k'}$ and add them to $\mathcal{N}_k$, for some $k' < k$. In contrast, updates in [8] are guaranteed not to remove transitions from any $\mathcal{N}_k$. Thus, a local state s_k cannot be confirmed in isolation, since its addition may remove transitions from some other $\mathcal{N}_{k'}$ that has already been (incorrectly) applied to a level- k' node below.

<pre> Enabled(in : level k, MDD p, MxD $\mathcal{N}$) 1: if $\mathcal{N} = 1$ then return $p \neq 0$; 2: if $\mathcal{N} = 0 \vee p = 0$ then return false; 3: if $\exists a : (k, p, \mathcal{N}, a) \in enabledCache$ then return a; 4: $a \leftarrow false$; 5: if $Top(\mathcal{N}) < k$ then • $\mathcal{N}$ skips this level 6: for each $i : p[i] \neq 0$ do 7: if Enabled($k - 1, p[i], \mathcal{N}$) then $a \leftarrow true$; 8: end for 9: else 10: for each $i : p[i] \neq 0$ do 11: for each $j : \mathcal{N}[i, j] \neq 0$ do 12: if Enabled($k - 1, p[i], \mathcal{N}[i, j]$) then 13: $a \leftarrow true$; 14: if $j \notin \mathcal{S}_k$ then $\mathcal{U}_k \leftarrow \mathcal{U}_k \cup \{j\}$; 15: end if 16: end for 17: end for 18: end if 19: $enabledCache \leftarrow enabledCache \cup (k, p, \mathcal{N}, a)$; 20: return a; </pre>	<pre> RecFire(in : level k, MDD p, MxD $\mathcal{N}$) 1: if $\mathcal{N} = 1 \vee k = 0$ then return p; 2: if $\exists s : (p, \mathcal{N}, s) \in fireCache[k]$ then return s; 3: $s \leftarrow$ new level-k MDD node; 4: if $Top(\mathcal{N}) < k$ then • $\mathcal{N}$ skips this level 5: for each $i : p[i] \neq 0$ do 6: $s[i] \leftarrow RecFire(k - 1, p[i], \mathcal{N})$; 7: end for 8: else 9: for each $i : p[i] \neq 0$ do 10: for each $j \in \mathcal{S}_k : \mathcal{N}[i, j] \neq 0$ do 11: $f \leftarrow RecFire(k - 1, p[i], \mathcal{N}[i, j])$; 12: $u \leftarrow Union(k - 1, f, s[j])$; 13: if $s[j] \neq u$ then $s[j] \leftarrow u$; 14: end for 15: end for 16: end if 17: if $s.down \neq [0, \dots, 0]$ then 18: Saturate(k, s); 19: end if 20: Reduce(k, s); 21: $fireCache[k] \leftarrow fireCache[k] \cup (p, \mathcal{N}, s)$; 22: return s; </pre>
<pre> Saturate(in : level k, MDD p) 1: $\mathcal{L} \leftarrow \{i : p[i] \neq 0 \wedge \exists \mathcal{N}_k[i, j] \neq 0\}$; 2: while $\mathcal{L} \neq \emptyset$ do 3: Select and remove i from $\mathcal{L}$; 4: for each $j : \mathcal{N}_k[i, j] \neq 0$ do 5: if Enabled($k - 1, p[i], \mathcal{N}_k[i, j]$) then 6: if $j \in \mathcal{S}_k$ then 7: $f \leftarrow RecFire(k - 1, p[i], \mathcal{N}_k[i, j])$; 8: $u \leftarrow Union(k - 1, f, p[j])$; 9: if $p[i] \neq u$ then 10: $p[i] \leftarrow u$; 11: $\mathcal{L} \leftarrow \mathcal{L} \cup \{j\}$; 12: end if 13: else 14: $\mathcal{U}_k \leftarrow \mathcal{U}_k \cup \{j\}$; • Confirm local state j 15: Update($1, k$); 16: $\mathcal{L} \leftarrow \mathcal{L} \cup \{i\}$; 17: end if 18: end if 19: end for 20: end while </pre>	<pre> Union(in : level k, MDD p, MDD q) 1: if $p = 0 \vee p = q$ then return q; 2: if $q = 0$ then return p; 3: if $\exists s : (p, q, s) \in uCache[k]$ then return s; 4: $s \leftarrow$ new level-k MDD node; 5: for each $i \in \mathcal{S}_k$ do 6: $s[i] \leftarrow Union(k - 1, p[i], q[i])$; 7: end for 8: Reduce($k, s$); 9: $uCache \leftarrow uCache \cup (p, q, s)$; 10: return s; Update(in : level k_{bot}, level k_{top}) 1: for $k \leftarrow k_{bot}$ to k_{top} do 2: if $\mathcal{U}_k \neq \emptyset$ then UpdateLevel(k); 3: end for 4: BuildAll($\mathcal{N}_K, \dots, \mathcal{N}_1$); </pre>

Figure 5. On-the-fly saturation algorithm for matrix diagrams

To this end, our algorithm delays exploration of newly-confirmed local state s_k by adding it to $\mathcal{U}_k$. In addition to solving the above problem, this approach also improves efficiency by reducing the number of times the overall MxDs must be reconstructed. The algorithm first confirms local states along a given MxD path (this is discussed in greater detail below); if any local states are confirmed, then the per-group next-state functions $\mathcal{N}_k^G$ are updated for any level k with $\mathcal{U}_k \neq \emptyset$ (using procedure UpdateLevel), and functions $\mathcal{N}_K, \dots, \mathcal{N}_1$ are reconstructed using BuildAll.

4.3. Generating the reachability set

Our modified version of saturation, which updates the next-state function and the local state spaces as necessary, is shown in procedure Saturate in Figure 5. Given a set of states p encoded as a level- k MDD node, Saturate updates the node to encode $(\mathcal{N}_k \cup \dots \cup \mathcal{N}_1)^*(p)$, where “*” denotes transitive closure. This is done recursively, by calling RecFire, which determines $\mathcal{N}(p)$ for a given next-state function $\mathcal{N}$ and set of states p and saturates any newly-created MDD

nodes. In [8], any unconfirmed local state that is reached by Saturate or RecFire is immediately confirmed. In our case, we instead use a separate preprocessing step to confirm local states: for a given set of states represented as an MDD p and a next-state function $\mathcal{N}$, procedure Enabled determines if the set $\mathcal{N}(p)$ is empty, and if not, adds any unconfirmed locals present in $\mathcal{N}(p)$ to the appropriate $\mathcal{U}_k$ set. If any $\mathcal{U}_k$ is nonempty, then the next-state functions are reconstructed by calling $\text{Update}(1, k)$, which updates levels 1 through k (via procedure UpdateLevel) and then rebuilds the next-state functions (via procedure BuildAll). Essentially, the procedure Enabled confirms any local states that would have been confirmed by RecFire ahead of time, allowing RecFire to saturate nodes with the correct, up-to-date next-state functions. Procedure RecFire is essentially the same as presented in [15]; it is shown in Figure 5 for completeness. Procedure Union, also shown in Figure 5, computes the union of two sets encoded as MDDs; details on this operation can be found in [16].

The saturation algorithm is invoked as follows. Sets $\hat{\mathcal{S}}_k$ and $\mathcal{U}_k$ should be initialized to $\mathcal{S}_k^0$, while $\mathcal{S}_k$ should be initially empty. The next-state functions can then be initialized by calling $\text{Update}(1, K)$. Next, the MDD representing the set of initial states $\mathcal{S}^0$ is constructed. If $\mathcal{S}^0 = \mathcal{S}_K^0 \times \dots \times \mathcal{S}_1^0$, then this MDD will contain one node at each level, and all downward pointers at level k will be zero except for pointers corresponding to $\mathcal{S}_k^0$, which will point to the level- $(k-1)$ node. The reachability set is then constructed by saturating all MDD nodes in $\mathcal{S}^0$ from the bottom up.

5. Implementation issues

In addition to the challenge of designing an efficient implementation of MDD nodes that can vary in size, as discussed in [8], the techniques described in earlier sections lead to some interesting implementation issues.

The matrix diagram nodes must be more dynamic in this case than as presented in [15]. The level- k nodes must support a matrix of downward pointers containing $|\hat{\mathcal{S}}_k|$ columns and $|\mathcal{S}_k|$ rows, where the sets $\hat{\mathcal{S}}_k$ and $\mathcal{S}_k$ are allowed to increase over time. In practice, this can be achieved using a sparse matrix representation based on linked-lists for each node, where missing entries are assumed to be to terminal node zero. In our implementation, MxD nodes for a given level are allowed to vary in size.

Problems can arise due to identity nodes when sets $\mathcal{S}_k$ increase. For instance, if a particular next-state function $\mathcal{N}$ skips level k , this implies that for the current $\mathcal{S}_k$, function $\mathcal{N}$ does not modify component k . However, if this is no longer true when $\mathcal{S}_k$ increases in size, then a level- k node must appear in $\mathcal{N}$. To avoid this issue, we do not remove identity nodes from group functions $\mathcal{N}_G^e$; this guarantees that the group functions will remain correct when local states are

added. Identity nodes are removed only when constructing functions $\mathcal{N}_K, \dots, \mathcal{N}_1$. Since these are rebuilt from functions $\mathcal{N}_G^e$ whenever a local state is confirmed, the resulting $\mathcal{N}_K, \dots, \mathcal{N}_1$ will be correct, as long as the MxD operation cache entries are cleared first. Specifically, when a local state is confirmed at level k , operation cache entries involving nodes at levels k and above must be removed.

During saturation at level k , an update can modify $\mathcal{N}_k$; care must be taken to ensure that the most up to date version is used. In particular, a transition that triggered an update might move from $\mathcal{N}_k$ to $\mathcal{N}_{k'}$ after the update. Indeed, after an update, it is possible that $\mathcal{N}_k = 0$. Thus, procedure Saturate must frequently check for updates to $\mathcal{N}_k$.

Note that a call to Saturate at level k can recursively call procedure RecFire at level $(k-1)$, which can call RecFire and Saturate at level $(k-2)$. It is possible for an update at level $(k-2)$ to change $\mathcal{N}_k$, which means that when execution returns to RecFire at level $(k-1)$, its passed MxD might not exist; we handle this by delaying the destruction of old MxD nodes until safe to do so. This is valid since the old MxD node used by RecFire at level $(k-1)$ still describes valid transitions within the context of the parameters passed to RecFire. Destruction of old MxD nodes is done in procedure Saturate: when called at level k , Saturate checks for updates to $\mathcal{N}_k$; if it has changed, the old $\mathcal{N}_k$ is destroyed and replaced with the newest version.

In [8], transitions can only be added to each $\mathcal{N}_k$ as local state spaces are increased; this implies that a level- k node p saturated before $\mathcal{N}_k$ increases is identical to the same node saturated after $\mathcal{N}_k$ increases. However, $\mathcal{N}_k$ can decrease in our case; if this occurs, a node saturated before $\mathcal{N}_k$ is updated may differ from the same node saturated after $\mathcal{N}_k$ is updated. Thus, whenever $\mathcal{N}_k$ changes, and the new $\mathcal{N}'_k$ is not a superset of the old $\mathcal{N}_k$, we must remove all entries from cache $\text{fireCache}[k']$ for all $k' \geq k$. The operation $\mathcal{N}_k \subseteq \mathcal{N}'_k$ can be performed using element-wise subtraction (which behaves like set difference[15]): if $\mathcal{N}_k - \mathcal{N}'_k$ is not zero, then some transition in $\mathcal{N}_k$ was removed during the update, and the caches must be cleared.

6. Experimental results

Prototypes of our algorithms have been implemented in SMART [6]. We test our approach on several models from the literature (described below), running on 933 Mhz Pentium III workstations with 512Mb of RAM, under the Linux operating system. For all models, we use the finest possible decomposition into components: each state variable (usually a Petri net place) becomes its own component. Results are shown in Table 1, including the number of tangible reachable states $|\mathcal{T}|$, the (final) number of MDD nodes required to represent $\mathcal{T}$, and performance measures for our pregeneration approach described in Section 3

Model	N	$ T $	Final nodes	Peak MDD nodes			CPU time (sec)			Peak cache		
				PRE	OTF	Kron	PRE	OTF	Kron	PRE	OTF	Kron
Kanban	20	8.05×10^{11}	696	14,619	14,094	24,067	2	4	4	56,830	86,122	82,448
	40	9.94×10^{14}	2,176	89,219	86,974	158,507	12	45	36	368,820	568,595	561,298
	80	1.56×10^{18}	7,536	610,419	601,134	1,142,587	145	758	775	2,622,400	4,101,717	4,116,198
FTMP	6	9.91×10^{15}	592	1,102	1,102	2,235	1	1	5	3,113	5,417	133,952
	25	5.32×10^{65}	8,876	19,532	19,532	29,215	4	19	985	55,933	96,845	9,786,104
	50	1.42×10^{131}	34,626	78,432	78,432	—	19	139	—	225,533	390,395	—
	100	1.00×10^{262}	136,751	314,357	314,357	—	111	1,130	—	905,983	1,568,120	—
Swaps	12	4.79×10^8	4,095	4,096	4,096	4,096	8	14	160	294,790	429,958	135,168
	14	8.72×10^{10}	16,383	16,384	16,384	16,384	60	98	1,828	1,605,462	2,350,934	745,472
	16	2.09×10^{13}	65,535	65,536	65,536	65,536	489	825	29,825	8,388,382	12,320,542	3,932,160
Courier	10	4.25×10^9	1,431	543,861	71,735	n/a	538	14	n/a	1,743,038	304,612	n/a
	20	2.26×10^{12}	4,191	2,150,924	227,230	n/a	7,119	82	n/a	7,008,138	857,572	n/a
	40	2.18×10^{15}	13,911	—	801,920	n/a	—	668	n/a	—	2,646,692	n/a
FMS	10	2.54×10^7	713	39,960	18,107	n/a	6	4	n/a	188,080	87,848	n/a
	20	8.83×10^9	2,213	202,740	77,712	n/a	101	31	n/a	1,090,505	362,788	n/a
	40	4.97×10^{12}	7,613	1,191,600	355,922	n/a	3,091	323	n/a	7,381,310	1,657,168	n/a

Table 1. Experimental results

(columns “PRE”), our on-the-fly approach described in Section 4 (columns “OTF”), and the on-the-fly, product-form approach described in [8] (columns “Kron”). For all approaches, we use a “lazy” garbage collection algorithm [7]. Performance measures we consider are the peak number of MDD nodes required, the peak total number of cache entries, and the total CPU time required to generate T (including the removal of vanishing states). Measures relating to MxD nodes are omitted for space; in summary, we found that the matrix diagram memory requirements are similar to the Kronecker representation memory requirements of [8].

6.1. Models

We first consider a model of a Kanban manufacturing system [16] which satisfies the Kronecker product requirement. Note that our approaches are quite competitive even with models satisfying the necessary requirements of [8]. In this case, our pregeneration algorithm has a different number of peak nodes than our on-the-fly algorithm; this is due to visitation order of local states within procedure Saturate (an update requires us to re-visit a local state).

Next we consider a fault-tolerant multiprocessor system (FTMP) presented for cases up to $N = 6$ in [11]; this emphasizes the efficiency of saturation. To satisfy the requirements for [8], events are split into several similar sub-events, each satisfying the Kronecker product requirement; this leads to increased computation time and cache size, since RecFire must be called separately for each event. Indeed, the cache memory requirements for [8] makes it inapplicable for the $N = 50$ and $N = 100$ cases.

The “Swaps” model represents an array of N distinct integers, with events to exchange two neighboring integers. To satisfy the Kronecker product requirement, each event must be split into N^2 sub-events, one for each possible state of the neighboring integers. This causes a dramatic increase in CPU time for the Kronecker approach [8]. Note that there are exactly $N!$ reachable states for this model.

The Courier model describes Courier protocol software [18] (using $N = M$), and the FMS model describes a flexible manufacturing system [9]. Both of these models contain immediate events, and thus cannot use the Kronecker approach [8]. The FMS model contains events that do not satisfy the Kronecker product requirement (i.e., we do not consider a simplified version of the model, as done in [15]).

6.2. Discussion

From Table 1, we can observe the following trends. For models “Kanban”, “FTMP”, and “Swaps”, we see that our pregeneration approach performs better than our on-the-fly approach. In these cases, pregeneration constructs the same local state spaces as on-the-fly, and the difference in computation time is due to the overhead of reconstructing the next-state MxDs during updates. The increased cache size for on-the-fly is due to the additional cache *enabledCache*. We also see that our approaches perform as good or better than the Kronecker approach [8]; this is especially true for models whose events do not satisfy the Kronecker product requirement and require splitting. Indeed, the strength of our approach is that events are essentially grouped together into K “macro-events”, where the MxD for each $\mathcal{N}_k$ is able to exploit similarities between the grouped events.

However, for models “Courier” and “FMS”, we instead see that the on-the-fly approach significantly outperforms the pregeneration approach. The primary reason for this is that the pregeneration approach constructs local state spaces that are slightly too large (since event priority is not considered during their construction), leading to unnecessarily larger and more complex MxDs. Complexity is exacerbated due to the corrections for priority.

Finally, we note that, to use the pregeneration approach it was necessary to modify models “Kanban”, “Courier”, and “FMS” (by hand, via addition of inhibitor arcs, implicit places, etc.) to obtain finite local state spaces.

7. Conclusion

We presented modifications to the saturation algorithm that allow its application to models whose events do not satisfy the Kronecker product requirement. We considered both pregeneration and on-the-fly construction of local state spaces and next-state functions. As expected, when pregeneration is able to construct exactly the same local state spaces and next-state functions, it performs better than on-the-fly; however, on-the-fly can perform significantly better than pregeneration in cases where the local state spaces cannot be accurately constructed in isolation.

While Kronecker-based saturation can still be applied to models whose events do not satisfy the Kronecker product requirement, doing so requires splitting events into sub-events that are expressible as Kronecker products. Our experiments indicate that the use of matrix diagrams instead can substantially reduce computation time in these cases.

References

- [1] R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.*, C-35(8):677–691, Aug. 1986.
- [2] R. E. Bryant. Symbolic boolean manipulation with ordered binary-decision diagrams. *ACM Computing Surveys*, 24(3):293–318, Sept. 1992.
- [3] P. Buchholz. Hierarchical structuring of superposed GSPNs. In *7th Int. Workshop on Petri Nets and Performance Models (PNPM’97)*, pages 81–90, Saint Malo, France, June 1997. IEEE Comp. Soc. Press.
- [4] J. Burch, E. Clarke, and D. Long. Symbolic model checking with partitioned transition relations. In A. Halaas and P.B. Denyer, editors, *Int. Conference on Very Large Scale Integration*, pages 49–58, Edinburgh, Scotland, Aug. 1991. IFIP Transactions, North-Holland.
- [5] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang. Symbolic model checking: 10^{20} states and beyond. *Information and Computation*, 98(2):142–170, June 1992.
- [6] G. Ciardo, R. Jones, A. Miner, and R. Siminiceanu. Logical and stochastic modeling with SMART. In P. Kemper and W. H. Sanders, editors, *Proc. 13th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, LNCS 2794, pages 78–97, Urbana, IL, USA, Sept. 2003. Springer-Verlag.
- [7] G. Ciardo, G. Luetgen, and R. Siminiceanu. Saturation: an efficient iteration strategy for symbolic state space generation. In T. Margaria and W. Yi, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2031, pages 328–342, Genova, Italy, Apr. 2001. Springer-Verlag.
- [8] G. Ciardo, R. Marmorstein, and R. Siminiceanu. Saturation unbound. In H. Garavel and J. Hatcliff, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2619, pages 379–393, Warsaw, Poland, Apr. 2003. Springer-Verlag.
- [9] G. Ciardo and K. S. Trivedi. A decomposition approach for stochastic reward net models. *Perf. Eval.*, 18(1):37–59, 1993.
- [10] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.
- [11] S. Derisavi, P. Kemper, and W. H. Sanders. Symbolic state-space exploration and numerical analysis of state-sharing composed models. In A. N. Langville and W. J. Stewart, editors, *4th Int. Conference on the Numerical Solution of Markov Chains (NSMC’03)*, pages 167–189, Urbana, IL, USA, Sept. 2003.
- [12] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [13] P. Kemper. Reachability analysis based on structured representations. In J. Billington and W. Reisig, editors, *Application and Theory of Petri Nets 1996 (Proc. 17th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1091, pages 269–288, Osaka, Japan, June 1996. Springer-Verlag.
- [14] A. S. Miner. Efficient solution of GSPNs using Canonical Matrix Diagrams. In R. German and B. Haverkort, editors, *9th Int. Workshop on Petri Nets and Performance Models (PNPM’01)*, pages 101–110, Aachen, Germany, Sept. 2001. IEEE Comp. Soc. Press.
- [15] A. S. Miner. Implicit GSPN reachability set generation using decision diagrams. *Perf. Eval.*, 56(1-4):145–165, Mar. 2004.
- [16] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 6–25, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [17] E. Pastor, O. Roig, J. Cortadella, and R. Badia. Petri net analysis using boolean manipulation. In R. Valette, editor, *Application and Theory of Petri Nets 1994 (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 815, pages 416–435, Zaragoza, Spain, June 1994. Springer-Verlag.
- [18] C. M. Woodside and Y. Li. Performance Petri net analysis of communications protocol software by delay-equivalent aggregation. In *4th Int. Workshop on Petri Nets and Performance Models (PNPM’91)*, pages 64–73, Melbourne, Australia, Dec. 1991. IEEE Comp. Soc. Press.